CodeShield AI

Beginner's Guide: Codebase Connections & System Flow

1. Welcome! Let's Understand the Analogy First

If you are new to software engineering or AI, this system might look complicated. Let's make it simple. Think of CodeShield AI as a **high-security automated vault** and code scanning as a **safety audit process**.

System Component	Real-World Analogy	What it does in CodeShield AI
main.py	The Reception Desk / Front Desk	Accepts requests from the UI or CLI and forwards them to the workers.
scanner/engine.py	The General Inspector	Scans code looking for obvious issues like weak passwords or SQL que
agents/orchestrator.py	The Team Supervisor (HAL)	Coordinates multiple specialized AI agents, telling them who works whe
ai_triage.py	The Security Analyst	Uses AI (like Gemini/GPT) to double-check if the security warnings are r
auto_fix.py	The Expert Carpenter	Generates safe, valid code patches to fix the security weaknesses.
database/json_db.py	The Filing Cabinet	Stores scan logs and results into simple JSON files in the scans/ directo

2. The System Architecture Diagram

Here is the map showing how all parts connect. When the User uploads a folder, it enters through the **Controller**, gets scanned in **Scanning**, gets verified by the **AI Swarm**, and finally logged in the **Database**.

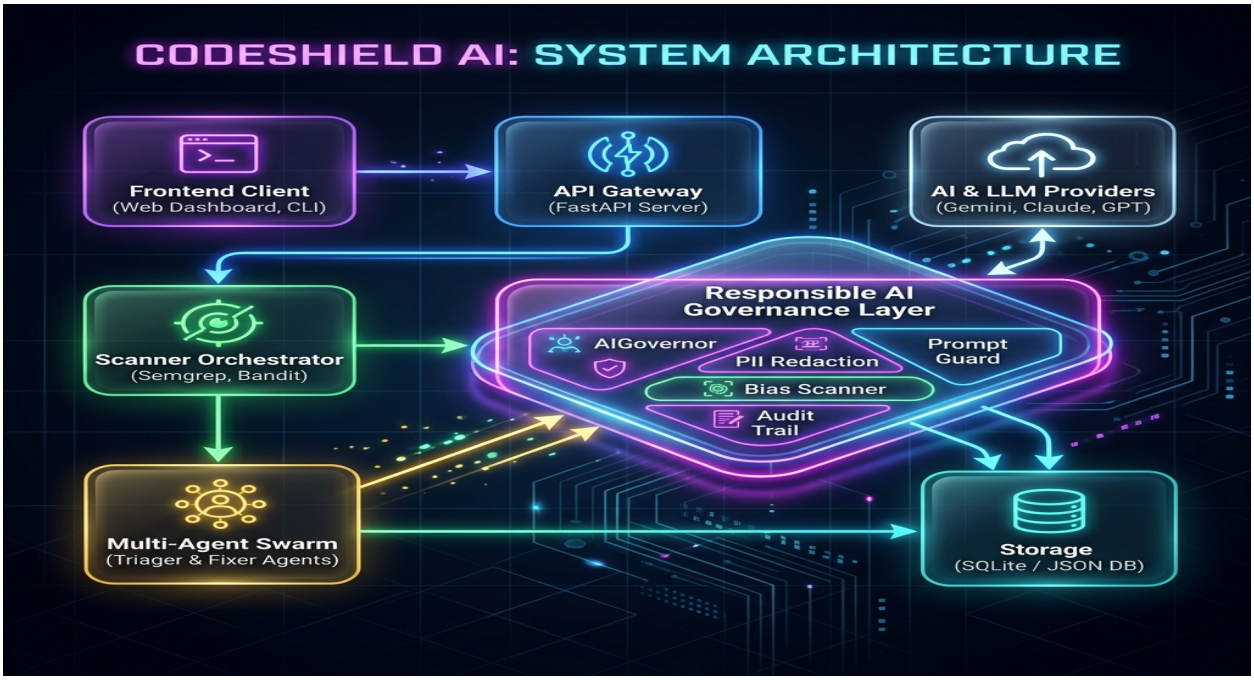

Figure 1: CodeShield AI System Architecture Map (Frontend -> Backend -> Swarm -> Database)

3. Step-by-Step Code Flow: How Files Talk to Each Other

Let's trace exactly what happens inside the codebase when you click **'Start Scan'** or run the CLI tool in your terminal. Follow these numbered steps:

Step 1: The Entrypoint (File: main.py)

The user submits a folder or ZIP file. FastAPI in **main.py** catches the request at POST /api/scan/zip, creates a unique **Scan ID**, and saves it in the database.

Step 2: Unpacking & Checking (File: scanner/engine.py)

main.py extracts the ZIP to a temporary directory and triggers the Scan Engine in **scanner/engine.py**. The engine detects what programming languages are in the folder.

Step 3: Finding Vulnerabilities (File: scanner/tools/)

scanner/engine.py runs security scanners like *Bandit* (for Python) or *ESLint* (for JavaScript) from the **scanner/tools/** folder. These scanners output raw findings.

Step 4: Starting the AI Swarm (File: agents/workflows.py)

Once the raw scan is completed, **main.py** starts the Multi-Agent Swarm in **agents/workflows.py** to check the results. The supervisor coordinates who inspects what.

Step 5: Verifying Real Warnings (File: ai_triage.py)

The swarm delegates warnings to **ai_triage.py**. This module uses an LLM (Gemini or OpenAI) to check the surrounding lines of code and filter out false alarms.

Step 6: Writing the Auto-Fix (File: auto_fix.py)

If the user requests a fix, the server calls **auto_fix.py**. This module asks the LLM to write a code patch, parses it into an AST (Abstract Syntax Tree) to verify it is secure and valid, and generates a code diff.

Step 7: Saving the Result (File: database/json_db.py)

Finally, all updates (verdicts, risk scores, progress, and fixes) are serialized and saved atomically into the scans directory by **database/json_db.py**, which the frontend displays to the user.

---

4. Beginner Glossary: Basic Security Concepts

Here are three basic terms you will see everywhere in this project:

1. **SAST (Static Application Security Testing):** Reading source code like a book to find bugs before running it (e.g., searching for hardcoded keys in a file).

2. **SCA (Software Composition Analysis):** Auditing project libraries and dependencies (e.g., checking package.json or requirements.txt) to see if you are using outdated, vulnerable third-party packages.
3. **Taint Analysis:** Tracking user input variables. If input variables flow into dangerous operations (like raw database queries) without sanitization, it flags a vulnerability.

Tip for Beginners: Start by looking at main.py to see how requests are accepted. Then, check database/json_db.py to see how data is written. Once you understand the basic FastAPI server flow, dive into agents/orchestrator.py to see how agents coordinate!